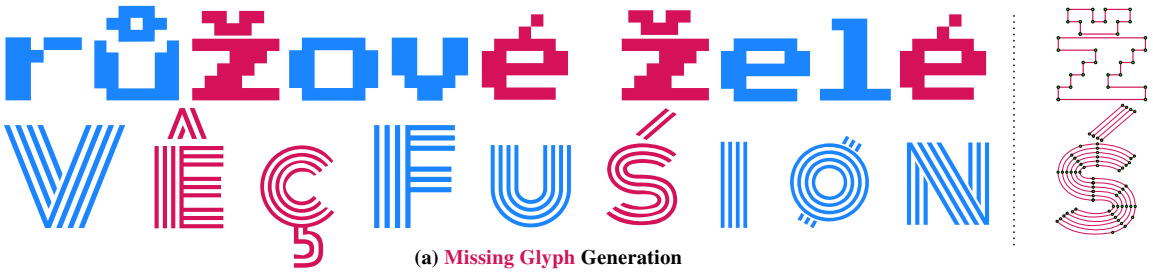
arXiv:2312.10540v2 [cs.CV] 21 May 2024

VecFusion: Vector Font Generation with Diffusion

Vikas Thamizharasan*[1,2] Difan Liu*[2] Shantanu Agarwal[1] Matthew Fisher[2]
Michaël Gharbi[2] Oliver Wang[3] Alec Jacobson[2,4] Evangelos Kalogerakis[1]

University of Massachusetts Amherst[1] Adobe Research[2] Google Research[3] University of Toronto[4]



(a) Missing Glyph Generation

Input Raster Images Synthesized Vector Glyphs

(b) Few-shot Vector Glyph Generation

Figure 1. We present VecFusion, a generative model for vector fonts. (a) VecFusion generates missing glyphs in incomplete fonts. Blue glyphs are glyphs that exist in the fonts. Red glyphs are missing glyphs generated by our method. On the right, we show generated control points as circles on selected glyphs. (b) VecFusion generates vector glyphs given a few exemplar (raster) images of glyphs. Our method generates precise, editable vector fonts whose geometry and control points are learned to match the target font style.

Abstract

We present VecFusion, a new neural architecture that can generate vector fonts with varying topological structures and precise control point positions. Our approach is a cascaded diffusion model which consists of a raster diffusion model followed by a vector diffusion model. The raster model generates low-resolution, rasterized fonts with auxiliary control point information, capturing the global style and shape of the font, while the vector model synthesizes vector fonts conditioned on the low-resolution raster fonts from the first stage. To synthesize long and complex curves, our vector diffusion model uses a transformer architecture and a novel vector representation that enables the modeling of diverse vector geometry and the precise prediction of control points. Our experiments show that, in contrast to previous generative models for vector graphics, our new cascaded vector diffusion model generates higher quality vector fonts, with complex structures and diverse styles.

1. Introduction

Vector fonts are extensively used in graphic design, arts, publishing, and motion graphics. As opposed to rasterized fonts, vector fonts can be rendered at any resolution without quality degradation, and can be edited via intuitive control point manipulation. However, authoring high-quality vector fonts remains a challenging and labor-intensive task, even for expert designers. Recent approaches [6, 31, 51] use VAEs or autoregressive models to automatically synthesize vector fonts, but they often struggle to capture a diverse range of topological structures and glyph variations, due to the inherent ambiguity of vector curves. As a result, they frequently create artifacts and imprecise control point positions, compromising the overall quality and editability of the synthesized fonts.

In this work, we leverage recent advances in *raster* generative models, to design a generative model for *vec-*

*Equal contribution
Project website: https://vikastmz.github.io/VecFusion/

tor fonts. Such a generative model has a number of real world applications, such as glyph completion, few-shot style transfer, and font style interpolation. However, training vector domain generative models is not straightforward: the irregular data structure of vector graphics prevents naive applications of commonly used CNN-based architectures. Furthermore, there is an inherent ambiguity in vector representations: infinitely many control points configurations can produce the same glyph, but not all configurations are equivalent. In particular, designers carefully create control points so that the font can be intuitively edited. Generated vector fonts should follow a similar design goal.

To address the above-mentioned challenges, we propose a novel two-stage diffusion model, called VecFusion, to generate high-quality vector fonts. Our pipeline is a cascade of a raster diffusion model followed by a vector diffusion model. The raster diffusion model gradually transforms a 2D Gaussian noise map to a target raster image of the glyph at low resolution, conditioned on a target glyph identifier and a target font style. It also generates auxiliary raster fields to drive the placement of vector control points in the next stage. The second stage is a vector diffusion model conditioned on the raster outputs from the first stage. It is trained to "denoise" a noisy vector glyph representation into structured curves representing the glyph.

Contributions. This papers makes several contributions. First, we present a novel two-stage cascaded diffusion model for high-quality vector fonts generation. This cascading process allows us to effectively "upsample" low-resolution raster outputs into a vector representation. We introduce a new mixed discrete-continuous representation for control points, which allows the vector diffusion model to automatically predict the number of control points and paths to use for a glyph, as well as their position. We show that diffusion models can effectively "denoise" in this new representation space. Moreover, to capture long-range dependencies and accommodate the irregular nature of vector representations, we introduce a transformer-based vector diffusion model. Finally, we show that VecFusion synthesizes fonts with much higher fidelity than state-of-the-art methods evaluated on datasets with diverse font styles.

2. Related Work

Generative vector graphics. Significant work has been invested in generative modeling of vector graphics, using VAEs [22, 31], sequence-to-sequence models like RNNs [14] or transformers [40]. Recent approaches employ hierachical generative models [6], while others bypass the need for direct vector supervision [39], using a differentiable rasterizer [25].

Font generation. Due to their ubiquity and central role in design, fonts have received special attention and dedicated synthesis methods. Many methods learn to generate *raster* fonts from a large set of reference glyphs [12, 19] or a few exemplar images [1, 7, 13, 23, 37, 45]. These methods produce visually appealing raster fonts in a variety of styles, but cannot generate vector outputs, thus they are limited by resolution and pixelization artifacts. In the task of *vector* font generation, early approaches use morphable template models [44], or manifold learning to enable interpolation/extrapolation of existing fonts [5], while recent methods use deep generative models [26, 50]. The first generation of deep learning solution sometimes generated glyphs with strong distortion and visual artifacts. Methods like DeepVecFont-v2 [51] improve the synthesis quality using a transformer architecture. Although these methods can generate visually pleasing vector fonts, effectively modeling a diverse distribution of glyphs and topologies remains a challenge. DeepVecFont-v2, for instance, only supports a limited number of glyphs (52 characters).

Diffusion models. To address the challenges in vector field design, we leverage diffusion models [15] for their ability to model diverse and complex data distributions. Unlike previous methods [9, 48] that use CNN or RNN-based vector diffusion models, our approach uses a transformer-based vector diffusion model to handle long-range dependencies inherent to complex vector glyphs. Furthermore, our two-stage raster–vector approach and novel vector representation enable precise Bézier curve prediction on challenging artist-designed font datasets.

Cascaded diffusion. Cascaded diffusion models [17] have achieved impressive synthesis quality across various domains, including images [2, 42], videos [4, 16] and 3D [18, 27]. In the same spirit, we introduce a cascaded diffusion model for high-quality vector font generation.

Image vectorization. Image vectorization approaches [3, 33, 46] output a vector graphics representation from a raster image. Dedicated for line drawing vectorization, many learning-based methods [21, 35, 38] have been proposed. Although these methods can produce high-quality vector graphics, they often create redundant or imprecise control points and fail to produce high-fidelity results on low-resolution raster images. Our diffusion model can generate precise vector geometry from low-resolution raster images, also providing a new perspective for image vectorization.

3. Method

Overview. The goal of VecFusion is to automatically generate vector graphics representations of glyphs. The input to our model is the Unicode identifier for a target character,

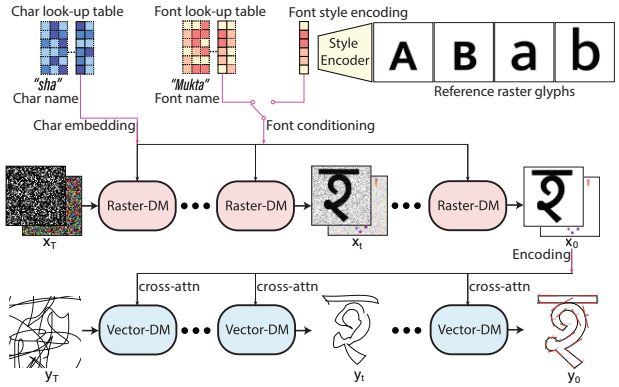


Figure 2. Overview of the VecFusion's cascade diffusion pipeline. Given a target character and font conditioning, our raster diffusion stage ("Raster-DM") produces a raster image representation of the target glyph in a series of denoising steps starting with a noise image. The raster image is encoded and input to our vector diffusion stage ("Vector-DM") via cross-attention. The vector diffusion stage produces the final vector representation of the glyph also in a series of denoising steps starting with a noise curve representation.

also known as *code point*, and a target font style. The target font style can be specified in the form of a few representative raster images of other glyphs in that style or simply by the font style name. Figure 2 shows an example of the generated vector representation for the glyph corresponding to the input target letter "sha" of the Devanagari alphabet and the target font style "Mukta". Our method is trained once on a large dataset of glyphs from various font styles. Once trained, it can generate glyphs not observed during training. Our model has several applications: generate missing glyphs in incomplete fonts, synthesize novel fonts by transferring the style of a few exemplar images of glyphs, or interpolate font styles.

Output vector representation. The generated vector representation for a glyph is in the form of ordered sequences of control points in cubic Bézier curve paths commonly used in vector graphics. Control points can be repeated in the generated sequences to manipulate the continuity of the vector path. Our method learns to generate an appropriate number of vector paths, control points, and point repetitions tailored to each character and font style. In addition, it learns the proper ordering of control points for each path, including where first and last control points are placed, since their placement patterns often reflect artist's preferences.

Pipeline. Our method consists of a two-stage cascade. In the first stage (raster diffusion model, or in short "Raster-DM", Figure 2), conditioned on the target character code point and font style, our method initiates a reverse diffusion process to generate a raster image. The generated raster image captures the shape and style of the target glyph at low resolution. Additionally, we generate an auxiliary set of *control point fields* encoding information for control point location, multiplicity, and ordering. In the second stage (vector diffusion model or "Vector-DM", Figure 2), our method proceeds by synthesizing the vector format capturing fine-grained placement of control points guided by the raster glyph and control point fields generated in the first stage. We observed that this two-stage approach results in generating higher-fidelity fonts compared to using diffusion in vector space directly or without any guidance from our control point fields. In the next sections, we discuss our raster and vector diffusion stages in more detail.

3.1. Raster diffusion stage

Given a target character identifier and font style, the raster diffusion stage creates a raster image $\mathbf{x}_0$ encoding information about the target glyph in pixel space (Figure 2, "Raster-DM"). This is performed through a diffusion model that gradually transforms an image $\mathbf{x}_T$ sampled from a unit Gaussian noise distribution towards the target raster image $\mathbf{x}_0$ in a series of T denoising steps. At each step $t = 1...T$, a trained neural network executes the transition $\mathbf{x}_t \rightarrow \mathbf{x}_{t-1}$ by predicting the noise content to be removed from the image $\mathbf{x}_t$. This denoiser network is conditioned on the input character identifier and font style. In the next paragraphs, we explain the encoding of the input character codepoint and font style, the target raster image, the denoiser network, and finally the training and inference processes of this stage.

Character identifier embeddings. Inspired by similar approaches in NLP to represent words [47], we create a one-hot vector representation for all unique character codepoints available in our dataset. Given a target character's codepoint, its one-hot vector representation is mapped to a continuous embedding $\mathbf{g}$ through a learned look-up table. The look-up table stores embeddings for all codepoints available in our dataset and retrieves them using the one-hot vector as indices.

Font style conditioning. To encode the font style, we experimented with two approaches depending on the application. To generate missing glyphs in incomplete fonts, we create a one-hot vector representation for all font styles available in our dataset. Given a target font style, its one-hot vector is mapped to a continuous embedding $\mathbf{f}$ through a learned look-up table as above. To generate glyphs conditioned on a few exemplar images, we concatenate the input images channel-wise and pass them through a convnet to get a font style feature map $\mathbf{f}$ (see supplement for more details.)

Target raster image. The target $\mathbf{x}_0$ produced in the raster diffusion stage is a $N \times N$ image made of the following channels:

(a) the first channel is composed of an image representing a grayscale rasterized image of the target glyph (Figure 3, top).
(b) the rest of the channels store *control point fields* (Figure 3, bottom), whose goal is to encode information about the control point location, multiplicity, and ordering. During training, this control point field is created by rendering each control point as a Gaussian blob centered at the 2D coordinates (x,y) of the control point. The coordinates are normalized in $[0, 1]^2$. We also modulate the color of the blob based on (a) the index of the control point in the sequence of control points of its vector path (e.g., first, second, third etc control point), and (b) its multiplicity. A look-up function is used to translate the ordering indices and multiplicities of control points to color intensities. In our implementation, we use 3 channels for this control point field, which can practically be visualized as an RGB image (Figure 3, bottom). These channels are concatenated with the raster image of the glyph, forming a 4-channel image.

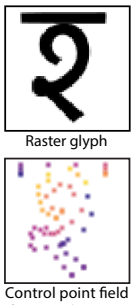


Figure 3. Target $\mathbf{x}_0$

Raster denoiser. The denoiser is formulated as a UNet architecture [11]. The network takes the 4-channel image $\mathbf{x}_t$ as input and is conditioned on the embedding of time step t. Following [41], we add the character's codepoint embedding $\mathbf{g}$ to time step embedding and pass it to each residual block in the UNet. For the font style conditioning, we add it to the time step embedding if it is a single embedding. If the font style is encoded as a spatial feature map, following [41], we flatten the feature map and inject it to the UNet via cross-attention.

The denoiser network predicts the per-channel noise component of the input image, which is also a 4-channel image (see supplement for more details.).

Training loss The network is trained to approximate an optimal denoiser under the condition that the images $\mathbf{x}_1, \mathbf{x}_2, ...\mathbf{x}_T$ are created by progressively adding Gaussian noise to the image of the previous step [15]: $q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}\big(\mathbf{x}_t; \sqrt{(1-\beta_t)}\mathbf{x}_{t-1}, \beta_t \mathbf{I}\big)$, where β_t represents the variance of the Gaussian noise added at each step. The image $\mathbf{x}_T$ converges to a unit Gaussian distribution as $T \rightarrow \infty$, or practically a large number of steps [15]. Following [15], we train the denoiser network with the training objective $||\boldsymbol{\epsilon}(\mathbf{x}_t, t, \mathbf{f}, \mathbf{g}) - \boldsymbol{\epsilon}||^2$ i.e. the mean-squared error loss between the added training noise $\boldsymbol{\epsilon}$ at each step and the predicted noise $\boldsymbol{\epsilon}(\mathbf{x}_t, t, \mathbf{f}, \mathbf{g})$ from the network. The loss is used to train the denoiser and the look-up tables.

Inference. At test time, given sampled unit Gaussian noise $\mathbf{x}_T$, a target character embedding $\mathbf{g}$ and font style conditioning $\mathbf{f}$, the network is successively applied in T steps to generate the target raster image.

Implementation details. In all our experiments, we use the following hyperparameters. Following [36], we set the number of diffusion steps T to 1000 and used cosine noise schedule in the forward diffusion process. Training takes 5 days on 8 A100 GPUs. We used the AdamW optimizer [32] with learning rate $3.24 \cdot 10^{-5}$. The feature embeddings for character identifiers are set to be 896-dimensional. The control points are rendered as Gaussian blobs with radius of 2 pixels. The raster image resolution is set to 64×64. Lower resolutions cause increasing overlaps between the rendered blobs, making the control point field more ambiguous. Increasing the resolution increases the computational overhead for the raster denoiser. The above resolution represented a good trade-off, as we practically found in our experiments. As mentioned above, we use 3 channels to encode control point ordering and multiplicity as colors. We practically observed that 3 channels were enough to guide the vector diffusion stage. Depending on the dataset, fewer channels could be used instead e.g., in cases of glyphs with few control points, or no multiplicities. In general, these hyperparameters can be adjusted for different vector graphics tasks – our main point is that the raster images and fields are useful as guidance to produce high-fidelity vector fonts, as shown in our ablation.

3.2. Vector diffusion stage

Given the raster image generated in the previous stage, the vector diffusion stage creates a tensor $\mathbf{y}_0$ representing the target glyph in vector graphics format (Figure 2 "Vector-DM"). The reverse diffusion process gradually transforms a noise tensor $\mathbf{y}_T$ sampled from a unit Gaussian noise distribution towards a tensor $\mathbf{y}_0$ in a series of denoising steps. In this domain, the noise represents noise on the *spatial position* and *path membership* of the control points, rather than the intensity of the pixel values as in the raster domain. In the next paragraphs, we explain the tensor representation, the denoiser, training and inference of this stage.

Target tensor. The target tensor $\mathbf{y}_0$ is a $M \times D$ tensor (Figure 4), where M represents an upper bound to the total number of control points a glyph can have. Each entry in the tensor contains a D-dimensional representation of a control point. Specifically, each entry stores the following information:
(a) the index of the vector path the control point belongs to i.e, its path membership. During training, each vector path is assigned a unique index. Since the vector paths can be re-ordered arbitrarily without changing the resulting glyph, to reduce unnecessary variability during learning, we lexigraphically sort vector paths using the coordinates of their

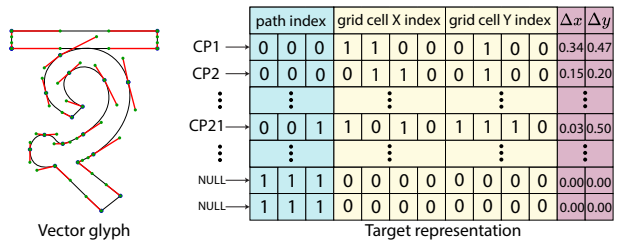


Figure 4. *Target tensor representation* $\mathbf{y}_0$. Our vector diffusion model "denoises" this tensor representation which includes both path membership and spatial position for control points. The discrete values (path membership, grid cell coordinates) are denoised in the continuous domain and then discretized. The control point locations are computed from the predicted grid cell coordinates plus continuous displacements $(\Delta x, \Delta y)$ from them.

control point closest to the top-left corner of the glyph raster image as sorting keys. Following [8], the resulting sorted path index is converted to binary bits. For each control point entry, we store the binary bits of its vector path. A null entry (i.e., all-one bits) is reserved for entries that do not yield control points – in this manner, we model vector fonts with a varying number of control points and paths.

(b) the index of the grid cell containing the control point. We define a coarse $P \times P$ grid over the image, with P^2 corresponding grid cell centroids. We assign each control point to the grid cell that has the closest centroid. In case the control point lies on the boundary of two cells, we use a round operation that assigns it in the second cell. Similar to path membership, the grid cell index is converted to binary bits. For each control point entry, we store the binary bits of its assigned grid cell.

(c) the continuous coordinates of the control point expressed relative to the center of the grid cell it belongs to. These are two continuous values capturing the location of each control point. We found that capturing control point locations relative to cell centers achieves the best performance. Since the generated raster control point field (approximately) highlights regions storing control points, mapping the control point field to discrete cell indices plus small continuous residuals, or displacements, is an easier task and reduces the continuous coordinate variability needed to be captured by the model.

Denoiser. The denoiser for this stage is formulated as an encoder-only transformer [10], which takes the tensor $\mathbf{y}_t$ as input and is conditioned on the embedding of time step t, and the generated raster image $\mathbf{x}_0$ from the raster diffusion model. We use a ConvNet to encode the raster image $\mathbf{x}_0$ into high-dimensional features, which are input to the transformer via cross-attention similar to [41]. The transformer predicts the noise content as a $M \times D$ tensor at each step.

Training loss. We train the denoiser network according to mean-squared error loss between training noise and predicted one at sampled time steps: $||\boldsymbol{\epsilon}(\mathbf{y}_t, \mathbf{x}_0, t) - \boldsymbol{\epsilon}||^2$.

Inference. At test time, given a sampled tensor $\mathbf{y}_T$ from unit Gaussian noise and a generated raster image $\mathbf{x}_0$ of the previous stage, the denoiser network is applied in a series of T steps to generate the target tensor $\mathbf{y}_0$. Following the Analog Bits approach [8], the discrete binary bits in the target tensor representation are modeled as real numbers. These are simply thresholded to obtain the final binary bits at inference time. Given the predicted path membership, we create a set of vector paths according to the largest generated control path index number. Each non-null entry in the generated tensor yields a control point. The control points are implicitly ordered based on their entry index. The location of the control point is defined as the coordinate center of the assigned cell in the generated tensor plus the predicted relative displacement. Given this generated information, we directly reconstruct the vector paths without requiring any further refinement or post-processing.

Implementation details. In our implementation, we set the upper bound for the number of control points to $M = 256$, which was sufficient for the datasets we experimented with. We use 3 bits to represent the path membership, which can support up to 7 distinct vector paths. This was also sufficient for the datasets in our experiments. We set P to 16, resulting in 256 grid cells which can be represented by 8 binary bits. Together with the two-dimensional relative displacement, the final dimension of our target tensor D is 13 in our experiments. Similar to our raster diffusion model, we set the number of diffusion steps T to 1000, used cosine noise schedule, and the AdamW optimizer [32] with learning rate $3.24 \cdot 10^{-5}$. Training is done separately from the raster diffusion stage and takes 5 days on 8 A100 GPUs. During testing, we use the DDPM sampler [15] with 1000 steps. Generating a glyph by executing both stages takes around 10 seconds on a A100 GPU.

4. Results

In this section, we present experiments in three different application scenarios for our method. In the first scenario, we address the problem of *missing glyph generation* for a font (Section 4.1). Many users often experience a frustrating situation when they select a font they prefer, only to discover that it lacks certain characters or symbols they wish to utilize. This issue is particularly prevalent when it comes to non-Latin characters and mathematical symbols. As a second application, we apply our method for *few-shot font style transfer* (Section 4.2), where the desired font is specified in the form of a few exemplar raster glyphs, and the goal is to generate vector glyphs in the same font. Finally, we discuss *interpolation of font styles* (Section 4.3) i.e., generate

glyphs whose style lies in-between two given fonts.

4.1. Generation of missing Unicode glyphs

Existing public datasets or benchmarks to evaluate glyph generation are limited to a specific alphabet (e.g., Latin). Below we discuss a new dataset for evaluating generation of glyphs across different languages, math symbols, and other signs common in the Unicode standard. Then we discuss comparisons, ablation study, metrics for evaluation, and results.

Dataset. We collected a new dataset of 1424 fonts from Google Fonts. The dataset contains $324K$ glyphs, covering 577 distinct Unicode glyphs in various languages (e.g., Greek, Cyrillic, Devanagari), math symbols and other signs (e.g. arrows, brackets, currency). We randomly partition the dataset into $314K$-$5K$-$5K$ glyphs for training, validation, and testing respectively.

Comparison. We compare with "ChiroDiff" [9], which applies diffusion models to generate Kanji characters as polylines. Their method uses a set-transformer [24] to obtain a latent embedding from a 2D point set as the input condition. We replace their input condition to their diffusion model with the embeddings of characters and fonts using look-up tables, as done in our raster diffusion model. We trained and tuned their method, including the embeddings, to predict Bézier curve control points using our dataset, as in our method.

Ablation. In addition, we evaluate the following alternative variants of our method: (a) ***Vector only:*** in this ablation, we remove the raster diffusion model and use the vector diffusion model only – in this case, the font and character embeddings are used as input conditions to the vector diffusion model. (b) ***No control point fields:*** we remove the RGB control point field from the target raster image of our raster diffusion model – in this case, we condition the vector diffusion model only on the single-channel raster image of the glyph. (c) ***Predict continuous coordinates only:*** in the vector diffusion model, instead of predicting discrete grid cell indices plus displacements relative to cell centers, we directly predict the absolute coordinates x and y per control point.

Evaluation metrics. We compare the generated glyphs with ones designed by artists in the test split. We use the following metrics:
(a) L_1: we compare the image-space absolute pixel differences of glyphs when rasterized. We use the same rasterizer for all competing methods and variants. This reconstruction error was also proposed in [50] for glyph evaluation.

Method	$L_1 \downarrow$	CD $\downarrow$	#cp diff $\downarrow$	#vp diff $\downarrow$
Ours	**0.014**	**0.16**	**3.05**	**0.03**
Ours (cont. coord. only)	0.020	0.18	3.30	**0.03**
Ours (no cp fields)	0.016	0.60	12.46	0.13
Ours (vector only)	0.016	0.68	9.36	0.11
ChiroDiff [9]	0.044	1.66	56.37	0.77

Table 1. Missing glyph generation evaluation on the full test set.

Method	$L_1 \downarrow$	CD $\downarrow$	#cp diff $\downarrow$	#vp diff $\downarrow$
Ours	**0.021**	**0.35**	**3.92**	**0.04**
Ours (cont. coord. only)	0.028	0.40	4.11	**0.04**
Ours (no cp fields)	0.026	0.83	12.90	0.16
Ours (vector only)	0.025	0.84	9.72	0.15
ChiroDiff [9]	0.072	3.63	71.97	1.02

Table 2. Missing glyph generation evaluation on a more challenging subset of our test set where test glyphs are from different glyph families compared to any glyphs in the training set.

Method	$L_1 \downarrow$	CD $\downarrow$	#cp diff $\downarrow$	#vp diff $\downarrow$
Ours	**0.069**	**0.46**	**15.05**	**0.033**
DeepVecFont-v2 [51]	0.098	1.05	34.84	0.052
DualVector [30]	0.197	1.37	25.69	0.428

Table 3. Few-shot font style transfer evaluation.

(b) *CD:* we measure the bidirectional Chamfer distance between artist-specified control points and generated ones.
(c) *#cp diff:* we measure the difference between the number of artist-made control points and predicted ones averaged over all paths.
(d) *#vp diff:* we measure the difference between the number of artist-specified vector paths and predicted ones.

For all the above measures, we report the averages over our test split. We propose the last three metrics for comparing glyphs in terms of the control point and path characteristics, which are more relevant in vector font design.

Quantitative Results. Table 1 shows the quantitative results for ChiroDiff and the alternative variants of our method on the full test set. The full version of our method outperforms ChiroDiff and our reduced variants on all evaluation metrics. We note that a glyph in one family variation e.g., italics might have a different variation in the same font, e.g., bold. Although two different family variations of the same glyph often have largely different control point locations and distributions, we create an even more challenging subset of the test set where we removed all test glyphs that had a different family variation in the same font within the training set. The results on this subset are reported in Table 2. Chirodiff and other variants still have much higher error than ours. This indicates that our two-stage approach, and the mixed discrete-continuous representation of vector paths along with the control point fields are all important to

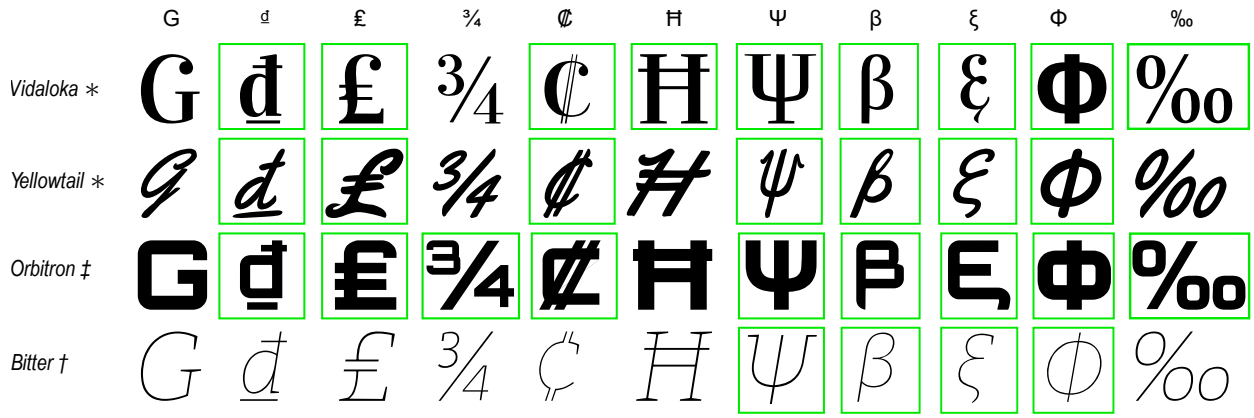


Figure 5. An incomplete font matrix from the Google Font dataset, each row represents a font and all glyphs in one column have the same Unicode. Glyphs in the green boxes are missing glyphs generated by our method. *: Regular, ‡: ExtraBold, †: ItalicVariableFontWidth.

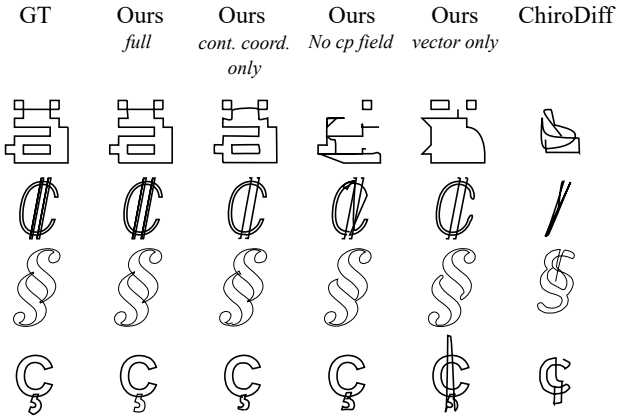


Figure 6. Glyph generation results for test cases from the Google font dataset. We compare our method to ChiroDiff [9] and degraded variants of our method. Our full method is able to generate glyphs that are much closer to artist-made ("ground-truth"/"GT") ones compared to alternatives.

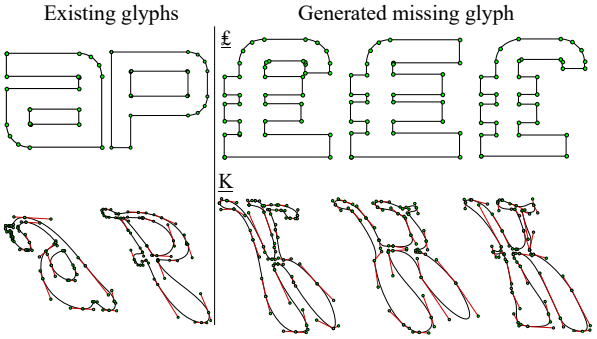


Figure 7. Stochastic sampling: the results are generated with three different random seeds.

achieve high performance.

Qualitative Results. Figure 6 shows qualitative comparisons for missing glyph generation. Compared to our method, we observe that ChiroDiff produces imprecise control points and curve structure, resulting in significant distortions and artifacts in the synthesized glyphs. We also observed degraded results in all alternative variants of our method in terms of misplaced or skipped control points. Additional results are also in the supplementary. Figures 1,5 show additional results of missing glyph generation for our method on the Google Font dataset for various target fonts and characters. Figure 7 demonstrates multiple samples generated by our diffusion pipeline with random seeds. The samples adhere to the same font style, while having subtle variation in the glyph proportions and control point distributions. From a practical point of view, a designer can explore multiple such samples, and choose the most preferred variant.

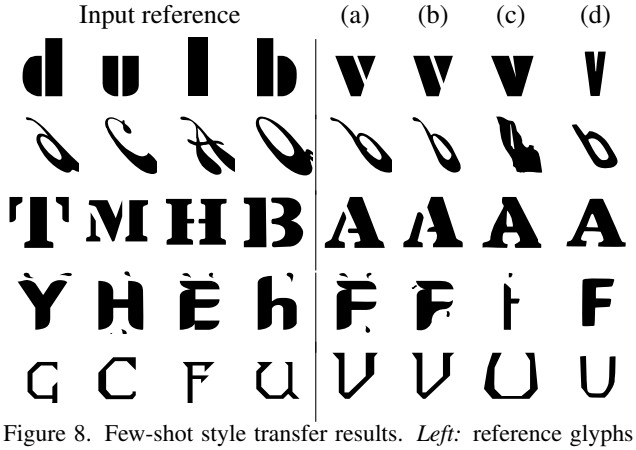


Figure 8. Few-shot style transfer results. *Left:* reference glyphs from a test font style. *Right:* (a) artist-made ("ground-truth") glyphs, (b) Ours, (c) DeepVecFont-v2 [51], and (d) DualVector [30].

In the supplementary, instead of using the vector diffusion model, we use off-the-shelf vectorization methods on the raster glyph image produced by the raster diffusion stage. As shown in the comparison, this approach often fails to produce coherent curve topology and structure.

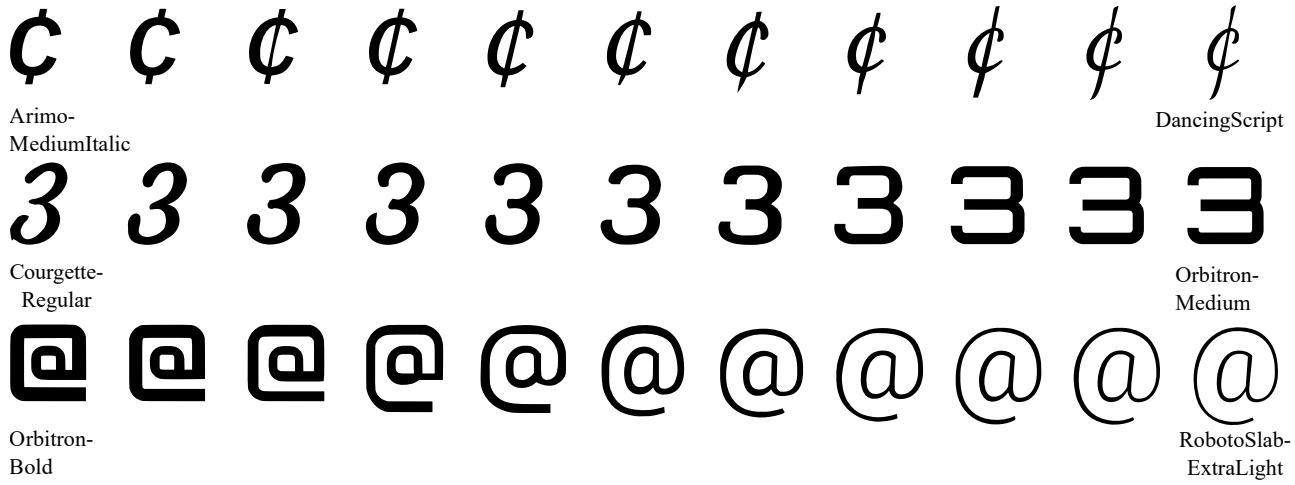


Figure 9. Font interpolation. We perform linear interpolation in embedding space from source font (left) → target font (right).

4.2. Few-shot font style transfer

For this application, we compare with DeepVecFont-v2 [51] and DualVector [30], both of which previously demonstrated few-shot font style transfer. To perform the comparison, we use the dataset proposed in the DeepVecFont-v2 paper. The dataset includes 52 lowercase and uppercase Latin characters in various font styles – there are total $8,035$ training fonts and $1,425$ test ones. Each test case contains 4 reference characters from a novel font (unobserved during training). The reference characters are available in both vector and raster format. Methods are supposed to transfer this novel font style to testing characters. We note that DeepVecFont-v2 requires the vector representation of the reference characters as additional input condition, while DualVector and our method only use the raster reference images.

Quantitative results. Table 3 shows numerical comparisons based on the same evaluation metrics as in the missing glyph generation application. Our method outperforms DeepVecFont-v2 and DualVector on all metrics.

Qualitative results. Figure 8 demonstrates font style transfer results. We observed that both DeepVecFont-v2 tends to succeed in capturing the font style of the reference glyphs, yet still often produces subtle distortions in the vector paths. Our method produces results that match the style of references glyphs with less artifacts, even in challenging styles. Additional results are in Figure 1 and in the supplementary.

4.3. Font style interpolation

Finally, we experimented with interpolating two given font styles. To perform interpolation, we first obtain the font emdeddings from our trained look-up table, then perform linear interpolation of these embeddings. Our diffusion model is then conditioned on the interpolated embedding vector for font style. We demonstrate qualitative results in Figure 9. Our results smoothly interpolates artistic properties of the source and target font, such as the variable stroke width and local curvature, while preserving structural and topological properties of the glyphs e.g., their genus.

5. Conclusion

We presented a generative model of vector fonts. We show that a cascade of a raster and vector diffusion model can overcome the challenges of neural parametric curve prediction, and generate editable vector fonts with precise geometry and control point locations.

Limitations. We require vector paths as supervision and cannot take advantage of raster images as additional supervision that may be available. Another limitation is shown in Figure 10. Cinzel is an uppercase font. For the missing glyph đ(dong), a lowercase glyph, our method "hallucinates" an uppercase glyph. While the generated glyph preserves the style of the font and the structure of the glyph (the stroke), it does not preserve the symbolic meaning of the glyph.

Figure 10. Failure case.

Future work. Generative models of images in the raster domain have been successfully used to learn complex priors about the visual world that can be used in various tasks such as inpainting and 3D tasks. We believe that a generative model for vector graphics in general could have several downstream applications, such as automatic generation and completion of logos, icons, line drawings and illustrations [20, 28, 29], or be extended to produce 3D parametric curves and surfaces [43].

Acknowledgments. Our project was funded by Adobe Research.

References

[1] Samaneh Azadi, Matthew Fisher, Vladimir G Kim, Zhaowen Wang, Eli Shechtman, and Trevor Darrell. Multi-content gan for few-shot font style transfer. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 7564–7573, 2018. 2

[2] Yogesh Balaji, Seungjun Nah, Xun Huang, Arash Vahdat, Jiaming Song, Karsten Kreis, Miika Aittala, Timo Aila, Samuli Laine, Bryan Catanzaro, et al. ediffi: Text-to-image diffusion models with an ensemble of expert denoisers. *arXiv preprint arXiv:2211.01324*, 2022. 2

[3] Mikhail Bessmeltsev and Justin Solomon. Vectorization of line drawings via polyvector fields. *ACM Trans. Graph.*, 38 (1):9:1–9:12, 2019. 2, 11, 12

[4] Andreas Blattmann, Robin Rombach, Huan Ling, Tim Dockhorn, Seung Wook Kim, Sanja Fidler, and Karsten Kreis. Align your latents: High-resolution video synthesis with latent diffusion models. *arXiv preprint arXiv:2304.08818*, 2023. 2

[5] Neill DF Campbell and Jan Kautz. Learning a manifold of fonts. *ACM Transactions on Graphics (ToG)*, 33(4):1–11, 2014. 2

[6] Alexandre Carlier, Martin Danelljan, Alexandre Alahi, and Radu Timofte. Deepsvg: A hierarchical generative network for vector graphics animation. *Advances in Neural Information Processing Systems*, 33:16351–16361, 2020. 1, 2

[7] Junbum Cha, Sanghyuk Chun, Gayoung Lee, Bado Lee, Seonghyeon Kim, and Hwalsuk Lee. Few-shot compositional font generation with dual memory. In *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XIX 16*, pages 735–751. Springer, 2020. 2

[8] Ting Chen, Ruixiang ZHANG, and Geoffrey Hinton. Analog bits: Generating discrete data using diffusion models with self-conditioning. In *The Eleventh International Conference on Learning Representations*, 2023. 5

[9] Ayan Das, Yongxin Yang, Timothy Hospedales, Tao Xiang, and Yi-Zhe Song. Chirodiff: Modelling chirographic data with diffusion models. In *International Conference on Learning Representations*, 2023. 2, 6, 7, 11, 13

[10] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018. 5, 11

[11] Prafulla Dhariwal and Alexander Nichol. Diffusion models beat gans on image synthesis. *Advances in Neural Information Processing Systems*, 34:8780–8794, 2021. 4, 11

[12] Yiming Gao and Jiangqin Wu. Gan-based unpaired chinese character image translation via skeleton transformation and stroke rendering. In *proceedings of the AAAI conference on artificial intelligence*, pages 646–653, 2020. 2

[13] Yue Gao, Yuan Guo, Zhouhui Lian, Yingmin Tang, and Jianguo Xiao. Artistic glyph image synthesis via one-stage few-shot learning. *ACM Transactions on Graphics (TOG)*, 38(6): 1–12, 2019. 2

[14] David Ha and Douglas Eck. A neural representation of sketch drawings. *arXiv preprint arXiv:1704.03477*, 2017. 2

[15] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *arXiv preprint arxiv:2006.11239*, 2020. 2, 4, 5

[16] Jonathan Ho, William Chan, Chitwan Saharia, Jay Whang, Ruiqi Gao, Alexey Gritsenko, Diederik P Kingma, Ben Poole, Mohammad Norouzi, David J Fleet, et al. Imagen video: High definition video generation with diffusion models. *arXiv preprint arXiv:2210.02303*, 2022. 2

[17] Jonathan Ho, Chitwan Saharia, William Chan, David J Fleet, Mohammad Norouzi, and Tim Salimans. Cascaded diffusion models for high fidelity image generation. *J. Mach. Learn. Res.*, 23(47):1–33, 2022. 2

[18] Ka-Hei Hui, Ruihui Li, Jingyu Hu, and Chi-Wing Fu. Neural wavelet-domain diffusion for 3d shape generation. In *SIGGRAPH Asia 2022 Conference Papers*, pages 1–9, 2022. 2

[19] Yue Jiang, Zhouhui Lian, Yingmin Tang, and Jianguo Xiao. Scfont: Structure-guided chinese font generation via deep stacked networks. In *Proceedings of the AAAI conference on artificial intelligence*, pages 4015–4022, 2019. 2

[20] Evangelos Kalogerakis, Derek Nowrouzezahrai, Patricio Simari, James McCrae, Aaron Hertzmann, and Karan Singh. Data-driven curvature for real-time line drawing of dynamic scene. *ACM Transactions on Graphics*, 28(1), 2009. 8

[21] Byungsoo Kim, Oliver Wang, A Cengiz Öztireli, and Markus Gross. Semantic segmentation for line drawing vectorization using neural networks. In *Computer Graphics Forum*, pages 329–338. Wiley Online Library, 2018. 2

[22] Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013. 2

[23] Yuxin Kong, Canjie Luo, Weihong Ma, Qiyuan Zhu, Shenggao Zhu, Nicholas Yuan, and Lianwen Jin. Look closer to supervise better: One-shot font generation via component-based discriminator. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 13482–13491, 2022. 2

[24] Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosiorek, Seungjin Choi, and Yee Whye Teh. Set transformer: A framework for attention-based permutation-invariant neural networks. In *International conference on machine learning*, pages 3744–3753. PMLR, 2019. 6

[25] Tzu-Mao Li, Michal Lukáč, Michaël Gharbi, and Jonathan Ragan-Kelley. Differentiable vector graphics rasterization for editing and learning. *ACM Transactions on Graphics (TOG)*, 39(6):1–15, 2020. 2

[26] Zhouhui Lian, Bo Zhao, Xudong Chen, and Jianguo Xiao. Easyfont: a style learning-based system to easily build your large-scale handwriting fonts. *ACM Transactions on Graphics (TOG)*, 38(1):1–18, 2018. 2

[27] Chen-Hsuan Lin, Jun Gao, Luming Tang, Towaki Takikawa, Xiaohui Zeng, Xun Huang, Karsten Kreis, Sanja Fidler, Ming-Yu Liu, and Tsung-Yi Lin. Magic3d: High-resolution text-to-3d content creation. *arXiv preprint arXiv:2211.10440*, 2022. 2

[28] Difan Liu, Mohamed Nabail, Aaron Hertzmann, and Evangelos Kalogerakis. Neural contours: Learning to draw lines

from 3d shapes. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020. 8

[29] Difan Liu, Matthew Fisher, Aaron Hertzmann, and Evangelos Kalogerakis. Neural strokes: Stylized line drawing of 3d shapes. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021. 8

[30] Ying-Tian Liu, Zhifei Zhang, Yuan-Chen Guo, Matthew Fisher, Zhaowen Wang, and Song-Hai Zhang. Dualvector: Unsupervised vector font synthesis with dual-part representation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. 6, 7, 8

[31] Raphael Gontijo Lopes, David Ha, Douglas Eck, and Jonathon Shlens. A learned representation for scalable vector graphics. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 7930–7939, 2019. 1, 2

[32] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017. 4, 5

[33] Xu Ma, Yuqian Zhou, Xingqian Xu, Bin Sun, Valerii Filev, Nikita Orlov, Yun Fu, and Humphrey Shi. Towards layer-wise image vectorization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16314–16323, 2022. 2

[34] Xu Ma, Yuqian Zhou, Xingqian Xu, Bin Sun, Valerii Filev, Nikita Orlov, Yun Fu, and Humphrey Shi. Towards layer-wise image vectorization. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2022. 11, 12

[35] Haoran Mo, Edgar Simo-Serra, Chengying Gao, Changqing Zou, and Ruomei Wang. General virtual sketching framework for vector line art. *ACM Transactions on Graphics (TOG)*, 40(4):1–14, 2021. 2

[36] Alexander Quinn Nichol and Prafulla Dhariwal. Improved denoising diffusion probabilistic models. In *International Conference on Machine Learning*, pages 8162–8171. PMLR, 2021. 4

[37] Song Park, Sanghyuk Chun, Junbum Cha, Bado Lee, and Hyunjung Shim. Multiple heads are better than one: Few-shot font generation with multiple localized experts. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 13900–13909, 2021. 2

[38] Ivan Puhachov, William Neveu, Edward Chien, and Mikhail Bessmeltsev. Keypoint-driven line drawing vectorization via polyvector flow. *ACM Transactions on graphics*, 40(6), 2021. 2

[39] Pradyumna Reddy, Michael Gharbi, Michal Lukac, and Niloy J Mitra. Im2vec: Synthesizing vector graphics without vector supervision. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 7342–7351, 2021. 2

[40] Leo Sampaio Ferraz Ribeiro, Tu Bui, John Collomosse, and Moacir Ponti. Sketchformer: Transformer-based representation for sketched structure. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 14153–14162, 2020. 2

[41] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10684–10695, 2022. 4, 5, 11

[42] Chitwan Saharia, William Chan, Saurabh Saxena, Lala Li, Jay Whang, Emily L Denton, Kamyar Ghasemipour, Raphael Gontijo Lopes, Burcu Karagol Ayan, Tim Salimans, et al. Photorealistic text-to-image diffusion models with deep language understanding. *Advances in Neural Information Processing Systems*, 35:36479–36494, 2022. 2

[43] Gopal Sharma, Difan Liu, Evangelos Kalogerakis, Subhransu Maji, Siddhartha Chaudhuri, and Radomír Měch. Parsenet: A parametric surface fitting network for 3d point clouds. In *Proceedings of the European Conference on Computer Vision*, 2020. 8

[44] Rapee Suveeranont and Takeo Igarashi. Example-based automatic font generation. In *Smart Graphics: 10th International Symposium on Smart Graphics, Banff, Canada, June 24-26, 2010 Proceedings 10*, pages 127–138. Springer, 2010. 2

[45] Licheng Tang, Yiyang Cai, Jiaming Liu, Zhibin Hong, Mingming Gong, Minhu Fan, Junyu Han, Jingtuo Liu, Errui Ding, and Jingdong Wang. Few-shot font generation by learning fine-grained local styles. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 7895–7904, 2022. 2

[46] Xingze Tian and Tobias Günther. A survey of smooth vector graphics: Recent advances in representation, creation, rasterization and image vectorization. *IEEE Transactions on Visualization and Computer Graphics*, 2022. 2

[47] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017. 3

[48] Qiang Wang, Haoge Deng, Yonggang Qi, Da Li, and Yi-Zhe Song. Sketchknitter: Vectorized sketch generation with diffusion models. In *The Eleventh International Conference on Learning Representations*, 2023. 2

[49] Xintao Wang, Liangbin Xie, Chao Dong, and Ying Shan. Real-esrgan: Training real-world blind super-resolution with pure synthetic data. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 1905–1914, 2021. 11, 12

[50] Yizhi Wang and Zhouhui Lian. Deepvecfont: Synthesizing high-quality vector fonts via dual-modality learning. *ACM Transactions on Graphics*, 40(6), 2021. 2, 6

[51] Yuqing Wang, Yizhi Wang, Longhui Yu, Yuesheng Zhu, and Zhouhui Lian. Deepvecfont-v2: Exploiting transformers to synthesize vector fonts with higher quality. *arXiv preprint arXiv:2303.14585*, 2023. 1, 2, 6, 7, 8, 11, 13

Appendix

A: Implementation Details

Here we provide additional implementation details of our network architecture. Our model is implemented in PyTorch.

Raster diffusion denoiser. Our raster diffusion denoiser follows the UNet architecture in [11, 41]. The UNet model uses a stack of residual layers and downsampling convolutions, followed by a stack of residual layers with upsampling convolutions, with skip connections connecting the layers with the same spatial size. We provide an overview of the hyperparameters in Table 4. To condition the model on the character identifier, we use a look-up table to project it to an 896-dimensional embedding and then add it together with the time step embedding to modulate the feature maps of each residual block.

Few-shot font style encoder. In the application of few-shot font style transfer, we used a ConvNet to encode reference raster glyphs into a font style feature map. We used the encoder part of the UNet architecture in [11, 41]. The ConvNet encoder encodes the 64×64 input image into an $8 \times 8 \times 512$ high-dimensional feature map via 3 downsampling layers.

Vector diffusion denoiser. Our vector diffusion denoiser is an encoder-only transformer following BERT [10]. We set the number of transformer layers and the number of attention heads to 8 and 12 respectively. To condition the vector diffusion denoiser on the raster guidance x_0, we first encode the $64 \times 64 \times 4$ raster image to a $16 \times 16 \times 768$ high-dimensional feature map with a ConvNet encoder. The ConvNet encoder has two downsampling layers with self-attention layers at resolution 32×32 and 16×16. The ConvNet encoder is trained with the transformer jointly. After obtaining the $16 \times 16 \times 768$ high-dimensional feature map, we flatten it to a shape of 256×768, then we add it to each transformer layer via cross-attention following [41].

Computation cost. Raster-DM and Vector-DM are trained separately. Each of them is trained on 8 A100 GPUs for 5 days. Finally, at inference time, generating a glyph takes around 10 seconds on a A100.

B: Additional Results

Comparison with a vectorizer approach. As an alternative comparison, we tried the following approach: instead of using the vector diffusion model, we use PolyVec [3] or LIVE [34] on the rasterized font image produced by our raster diffusion stage. We also tried upsampling the 64×64 output raster image to 256×256 using ESRGAN [49] before passing it to the vectorizer. We show qualitative comparison in Figure 11. In both cases, PolyVec and LIVE often failed to produce coherent curve topology, structure, and plausible control point distributions.

Input shape	$64 \times 64 \times 4$
Diffusion steps	1000
Noise Schedule	cosine
Channels	224
Depth	2
Channel Multiplier	1,2,3,4
Attention resolutions	32,16,8
Head Channels	32
Batch Size	448
Learning Rate	3.24e-5

Table 4. Hyperparameters for raster diffusion denoiser

Additional comparisons with DeepVecFont-v2 [51]. Please see Figure 12 for more comparisons with DeepVecFont-v2 [51] on the task of few-shot font style transfer.

Additional comparisons with ChiroDiff [9]. Please see Figure 13 for more comparisons with ChiroDiff [9] on the task of missing unicode generation.

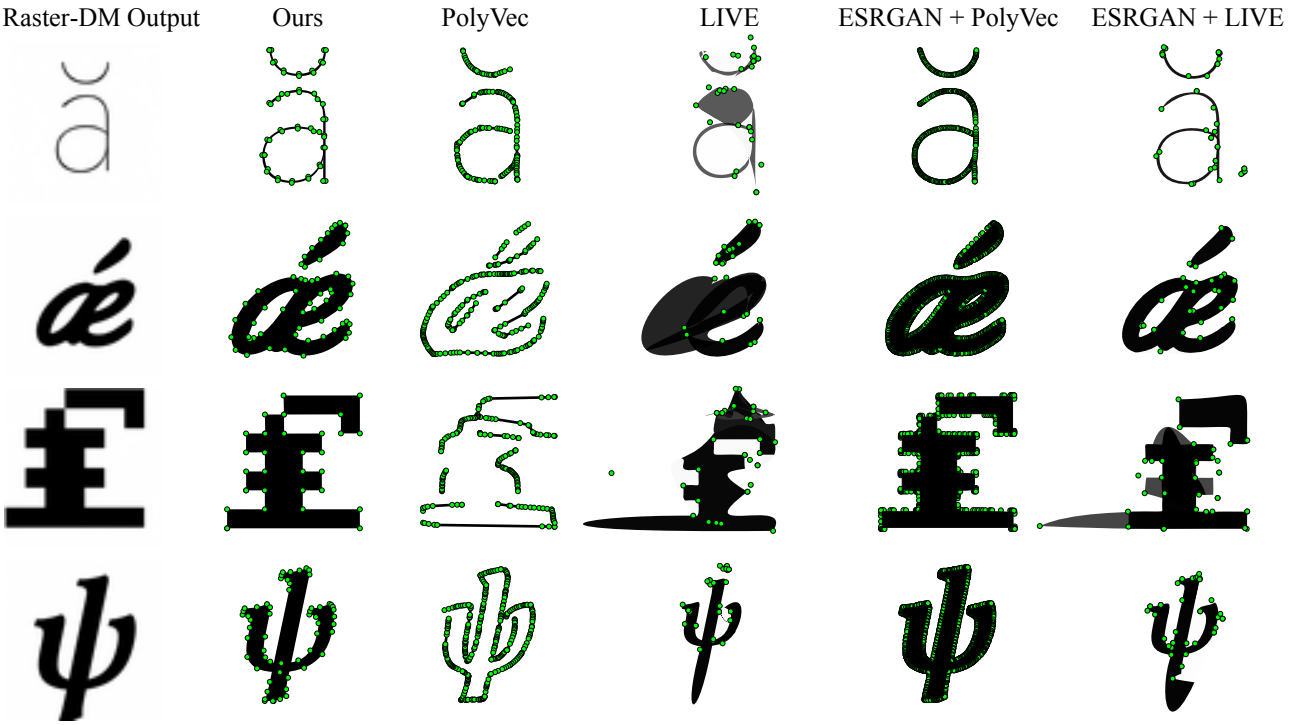


Figure 11. We compare our results (Ours) with PolyVec [3] and LIVE [34] applied to the raster image produced by our raster diffusion stage (left-most column). We also compare with PolyVec and LIVE applied to a higher-resolution version of the raster image upsampled via ESRGAN [49]. For each glyph, we show the predicted control points as well. Using our vector diffusion stage instead of an off-the-shelf vectorizer produces higher-quality glyphs and much more plausible control point distributions. Compared to our vector diffusion model, ESRGAN + PolyVec requires about ten times more control points for effective glyph reconstruction but sacrifices user editability and SVG compactness. We recommend the viewer to zoom in for better clarity.

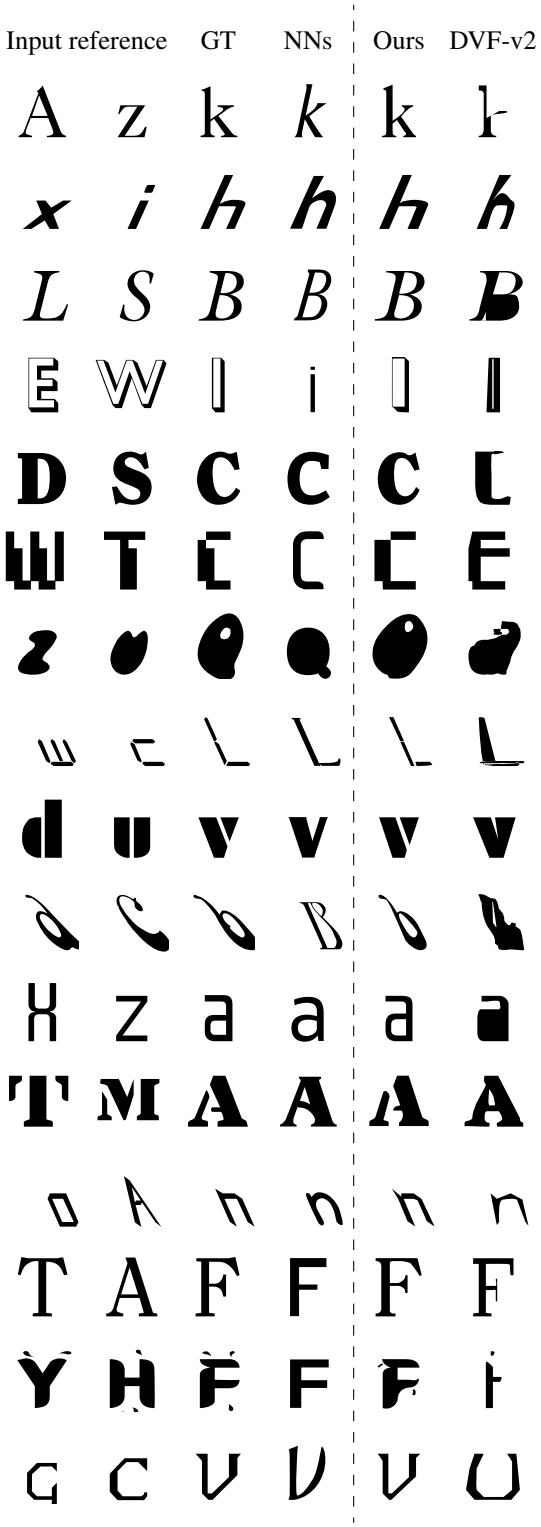


Figure 12. Few-shot style transfer results. From left to right, we show the reference glyphs (2 out of 4) belonging to a novel font style, the artist-made ("ground-truth/ GT") glyphs, the nearest-neighbours ("NNs") to GT in the training data, our generated ones, and DeepVecFont-v2 (DVF-v2) [51]

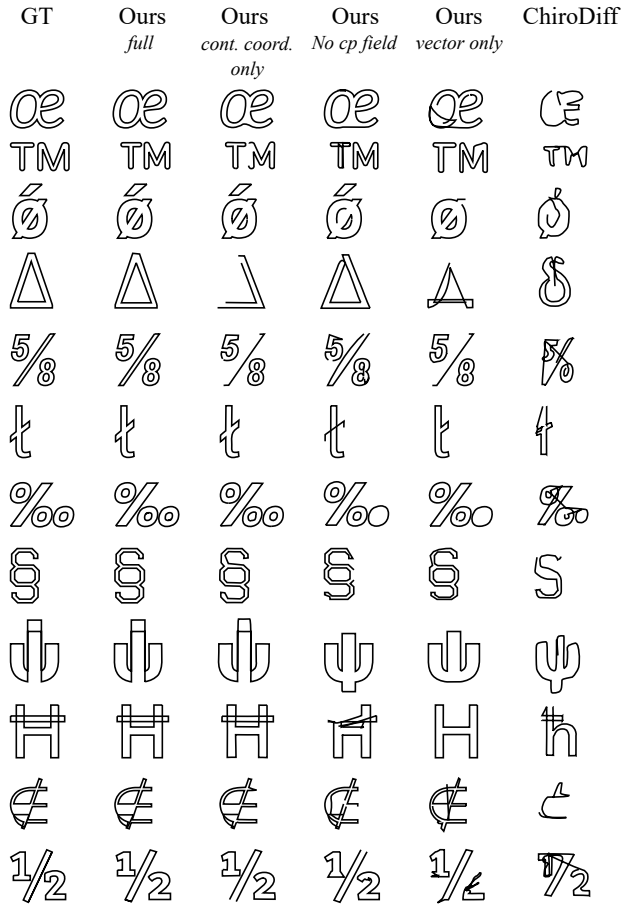


Figure 13. Glyph generation results for test cases from the Google font dataset. We compare our method to ChiroDiff [9] and degraded variants of our method. Our full method is able to generate glyphs that are much closer to artist-made ("ground-truth"/"GT") ones compared to alternatives.